

Aprendizaje estadístico. 2a parte. Tarea 1

Predicción con redes neuronales

Alex Sanchez

2026-05-01

Contents

Presentación	1
Descripción del problema	1
Datos para el entrenamiento	1
Tareas	2
Forward propagation (Iteración 1)	2
Cálculo de la función de pérdida	2
Backpropagation (Iteración 1)	2
Segunda iteración	3
Implementación	3
Entrenamiento completo de la red	3
Entregables	3

Presentación

El objetivo de este ejercicio es comprender cómo aprende una red neuronal, combinando:

- cálculos numéricos (a mano)
- implementación en Python o R

Descripción del problema

Consideraremos una red neuronal *feedforward* con:

- **2 variables de entrada** x_1, x_2
- **1 capa oculta con 2 neuronas**
- **1 neurona de salida**

Todas las neuronas utilizan la **función de activación sigmoidea**.

Datos para el entrenamiento

La primera parte es visualizar como se entrena la red por lo que debéis definir

- Valores iniciales de pesos y sesgos (bias).
- Valor de la tasa de aprendizaje η

Cada grupo deberá inicializar los pesos y sesgos de forma aleatoria utilizando una semilla propia.

Se recomienda usar como semilla un identificador único (por ejemplo, DNI o número de grupo).

Los pesos deben generarse en el intervalo $[-0.2, 0.2]$ y los sesgos en $[-0.05, 0.05]$.

Tened en cuenta que valores pequeños de los pesos ayudan a evitar la saturación de la función sigmoide al inicio del entrenamiento.

- Una única observación bivalente extraída aleatoriamente del conjunto de datos **Admissions-csv**

$$(x_1, x_2, y)$$

Tareas

Utilizad las fórmulas siguientes para el cálculo de cada paso de la red:

Forward propagation (Iteración 1)

Calcula manualmente:

- Capa oculta:
 - preactivaciones $z^{(1)}$
 - activaciones $a^{(1)}$
- Capa de salida:
 - preactivación $z^{(2)}$
 - predicción $\hat{y}$

Cálculo de la función de pérdida

Utilizamos como función de pérdida la **binary cross-entropy**:

$$\mathcal{L}(y, \hat{y}) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Backpropagation (Iteración 1)

Calcula paso a paso:

- Error en la capa de salida:

$$\delta^{(2)} = \hat{y} - y$$

Esta expresión es válida al combinar la función sigmoide con la cross-entropy.

- Error en la capa oculta:

$$\delta^{(1)} = (W^{(2)})^T \delta^{(2)} \cdot \sigma'(z^{(1)})$$

Donde, para la función sigmoidea se tiene que:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

Actualización de pesos

Con los valores calculados podemos actualizar todos los pesos y sesgos usando:

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}, \quad b \leftarrow b - \eta \frac{\partial L}{\partial b}$$

Segunda iteración

Repetid los pasos 1–4 utilizando los parámetros actualizados. Verificad que la función de pérdida disminuye.

Implementación

Utilizando el notebook proporcionado (basado en el ejemplo de Iris):

- Adaptad el código para reproducir los mismos cálculos
- Verificad que:
 - los resultados coinciden con los cálculos manuales
 - la función de pérdida disminuye tras la segunda iteración

Entrenamiento completo de la red

- Aplicad el mismo procedimiento al conjunto de datos `Admissions.csv`
- Entrenad la red durante varias iteraciones.
- Representad la evolución de la función de pérdida

Entregables

Cada grupo debe entregar, usando la tarea de Atenea, **un único informe** con los nombres de todos los miembros del grupo claramente visibles en la cabecera del trabajo.

Dicho informe debe contener

- Los cálculos numéricos claramente explicados. Debe poderse seguir paso a paso el proceso de entrenamiento en al menos una iteración completa.
- El entrenamiento del modelo completo y la valoración del modelo resultante,
- El código como un *notebook* de R o de Python, limpio y bien comentado. Este código debe poder reproducir el ejercicio en un ordenador independiente del vuestro.